# GPU 가속 시스템 상세 분석

## 개요

Proof of SQL의 GPU 가속 시스템은 Blitzar 프레임워크를 기반으로 하여 암호학적 연산을 CUDA 커널에서 병렬 처리함으로써 대폭적인 성능 향상을 달성합니다. 특히 commitment 계산과 다변수 스칼라 곱셈(Multi-Scalar Multiplication, MSM) 연산에서 GPU의 병렬 처리 능력을 활용하여 CPU 대비 수십 배의 성능 개선을 제공합니다.

## Blitzar 프레임워크 통합

### 의존성 구성

**Cargo.toml 설정:**

```
# workspace dependencies

blitzar = { version = "4.3.0" }



# proof-of-sql crate features

[features]

default = ["arrow", "perf"]

blitzar = ["dep:blitzar", "dep:merlin", "std"]

perf = ["blitzar", "cpu-perf"]
```

**라인별 분석:**

- `blitzar = { version = "4.3.0" }` : Blitzar 프레임워크의 정확한 버전 지정
- `blitzar = ["dep:blitzar", "dep:merlin", "std"]` : GPU 가속 기능 활성화 시 필요한 의존성들
- `perf = ["blitzar", "cpu-perf"]` : 성능 최적화 기능에 GPU 가속 포함

### 조건부 컴파일

**commitment/mod.rs:**

```
#[cfg(feature = "blitzar")]
pub use blitzar::{
compute::{init_backend, init_backend_with_config, BackendConfig},
proof::InnerProductProof,
};
```

**라인별 분석:**

- `#[cfg(feature = "blitzar")]` : blitzar 기능이 활성화된 경우에만 컴파일
- `init_backend` : GPU 백엔드 초기화 함수
- `init_backend_with_config` : 사용자 정의 설정으로 GPU 백엔드 초기화
- `BackendConfig` : GPU 백엔드 설정 구조체
- `InnerProductProof` : 내적 증명을 위한 GPU 가속 구조체

## GPU 초기화 과정

### Hello World 예제에서의 초기화

**examples/hello_world/main.rs:**

```
#[cfg(feature = "blitzar")]
{
let timer = start_timer("Warming up GPU");
proof_of_sql::base::commitment::init_backend();
end_timer(timer);
}
```

**라인별 분석:**

- `#[cfg(feature = "blitzar")]` : GPU 가속 기능이 활성화된 경우에만 실행
- `start_timer("Warming up GPU")` : GPU 초기화 시간 측정 시작
- `init_backend()` : CUDA 커널 로딩 및 GPU 메모리 할당
- `end_timer(timer)` : 초기화 완료 시간 출력 (일반적으로 ~520ms)

### 초기화 과정의 세부 단계

1. **CUDA 드라이버 검증**: GPU 드라이버와 CUDA 런타임 호환성 확인
2. **커널 로딩**: 미리 컴파일된 CUDA 커널들을 GPU 메모리에 로드
3. **메모리 풀 할당**: 대용량 연산을 위한 GPU 메모리 풀 사전 할당
4. **스트림 생성**: 비동기 연산을 위한 CUDA 스트림 초기화
5. **벤치마크 실행**: GPU 성능 측정 및 최적 설정 결정

## HyperKZG Commitment의 GPU 구현

### CPU vs GPU 구현 분기

**hyperkzg/commitment.rs:**

```rust
impl Commitment for HyperKZGCommitment {
type Scalar = BNScalar;
type PublicSetup<'a> = HyperKZGPublicSetup<'a>;


#[cfg(not(feature = "blitzar"))]
#[tracing::instrument(name = "compute_commitments (cpu)", level = "debug", skip_
fn compute_commitments(
committable_columns: &[crate::base::commitment::CommittableColumn],
offset: usize,
setup: &Self::PublicSetup<'_>,
) -> Vec<Self> {
compute_commitments_impl(committable_columns, offset, setup)
}


#[cfg(feature = "blitzar")]
#[tracing::instrument(name = "compute_commitments (gpu)", level = "debug", skip_
fn compute_commitments(
committable_columns: &[crate::base::commitment::CommittableColumn],
offset: usize,
setup: &Self::PublicSetup<'_>,
) -> Vec<Self> {
// GPU 구현...
```

```
}

}
```

**라인별 분석:**

- `#[cfg(not(feature = "blitzar"))]` : GPU 가속이 비활성화된 경우 CPU 구현 사용
- `#[cfg(feature = "blitzar")]` : GPU 가속이 활성화된 경우 GPU 구현 사용
- `#[tracing::instrument]` : 성능 추적을 위한 계측 매크로
- `name = "compute_commitments (gpu)"` : GPU 구현임을 명시하는 추적 이름

## GPU 구현의 핵심 로직

```
#[cfg(feature = "blitzar")]

fn compute_commitments(

committable_columns: &[crate::base::commitment::CommittableColumn],

offset: usize,

setup: &Self::PublicSetup<'_>,

) -> Vec<Self> {

if committable_columns.is_empty() {

return Vec::new();

}


// 최대 컬럼 길이를 찾아 생성원 개수 결정

let max_column_len = committable_columns

.iter()
```

```
.map(CommittableColumn::len)

.max()

.expect("You must have at least one column");


let mut blitzar_commitments = vec![G1Affine::default(); committable_columns.len(


blitzar::compute::compute_bn254_g1_uncompressed_commitments_with_generators(

&mut blitzar_commitments,

&slice_ops::slice_cast(committable_columns),

&setup[offset..offset + max_column_len],

);


slice_ops::slice_cast(&blitzar_commitments)

}
```

**라인별 분석:**

- `if committable_columns.is_empty()` : 빈 컬럼 배열에 대한 조기 반환
- `max_column_len` : 모든 컬럼 중 최대 길이 계산 (GPU 메모리 할당 크기 결정)
- `vec![G1Affine::default(); committable_columns.len()]` : 결과 저장용 버퍼 사전 할당
- `compute_bn254_g1_uncompressed_commitments_with_generators` : Blitzar의 핵심 GPU 함수 호출
- `slice_ops::slice_cast` : 타입 변환을 통한 효율적인 메모리 접근

## Blitzar 메타데이터 테이블 시스템

## 메타데이터 테이블 생성

**dory/blitzar_metadata_table.rs:**

```
#[tracing::instrument(name = "create_blitzar_metadata_tables", level = "debug",

pub fn create_blitzar_metadata_tables(

committable_columns: &[CommittableColumn],

offset: usize,

) -> (Vec<u32>, Vec<u32>, Vec<u8>) {

if committable_columns.is_empty() {

return (vec![], vec![], vec![]);

}


log::log_memory_usage("Start");


// 서명된 데이터 컬럼 처리를 위한 컬럼 길이 추적

let ones_columns_lengths = committable_columns

.iter()

.map(CommittableColumn::len)

.collect_vec();


// 스칼라 벡터 생성을 위한 최대 매트릭스 크기 계산

let (max_height, max_width) = if let Some(max_column_len) =

committable_columns.iter().map(CommittableColumn::len).max()
```

```
{
matrix_size(max_column_len, offset)
} else {
(0, 0)
};
// ... 추가 구현
}
```

**라인별 분석:**

- `#[tracing::instrument]` : 메타데이터 테이블 생성 과정 추적
- `log::log_memory_usage("Start")` : 메모리 사용량 로깅 시작
- `ones_columns_lengths` : 서명된 데이터 타입 처리를 위한 길이 정보
- `matrix_size(max_column_len, offset)` : GPU 연산에 최적화된 매트릭스 차원 계산

## 비트 테이블과 길이 테이블 생성

```
// 모든 committable 컬럼의 단일 패킹된 바이트 크기 계산
let num_of_bytes_in_committable_columns: usize = committable_columns
.iter()
.map(|column| column.column_type().byte_size())
.sum();

// 서명된 모든 committable 컬럼에 대해 ones가 추가된 단일 비트 테이블 엔트리
let single_entry_in_blitzar_output_bit_table: Vec<u32> = committable_columns
```

```
.iter()
.map(|column| column.column_type().bit_size())
.chain(core::iter::repeat_n(BYTE_SIZE, ones_columns_lengths.len()))
.collect();


// Blitzar의 vlen_msm 알고리즘에서 사용할 전체 비트 테이블 벡터 생성
let blitzar_output_bit_table: Vec<u32> = single_entry_in_blitzar_output_bit_tabl
.iter()
.copied()
.cycle()
.take(single_entry_in_blitzar_output_bit_table.len() * offset_height)
.collect();
```

**라인별 분석:**

- `byte_size()` : 각 데이터 타입의 바이트 크기 계산
- `bit_size()` : 각 데이터 타입의 비트 크기 계산
- `chain(core::iter::repeat_n(BYTE_SIZE, ones_columns_lengths.len()))` : 서명된 컬럼을 위한 추가 비트 할당
- `cycle().take()` : 반복 패턴을 통한 효율적인 테이블 생성

# 성능 최적화 기법

## 메모리 관리 최적화

```
// 스칼라 배열 생성. 스칼라는 컬럼-메이저 순서로 저장되어야 함
```

```
let num_scalar_rows = max_width;

let num_scalar_columns =

(num_of_bytes_in_committable_columns + ones_columns_lengths.len()) * offset_heig

let mut blitzar_scalars = vec![0u8; num_scalar_rows * num_scalar_columns];



// 스칼라 배열 채우기

let span = span!(Level::DEBUG, "pack_blitzar_scalars").entered();

if !blitzar_scalars.is_empty() {

if_rayon!(

blitzar_scalars.par_chunks_exact_mut(num_scalar_columns),

blitzar_scalars.chunks_exact_mut(num_scalar_columns)

)

.enumerate()

.for_each(|(scalar_row, scalar_row_slice)| {

// 병렬 처리 로직...

});

}
```

**라인별 분석:**

- `num_scalar_rows * num_scalar_columns` : GPU 메모리 레이아웃에 최적화된 크기 계산
- `vec![0u8; ...]` : 대용량 메모리 사전 할당으로 재할당 오버헤드 방지
- `span!(Level::DEBUG, "pack_blitzar_scalars")` : 세부 성능 추적
- `if_rayon!` : CPU 병렬 처리와 GPU 병렬 처리의 하이브리드 최적화

### 배치 처리 최적화

```
blitzar::compute::compute_bn254_g1_uncompressed_commitments_with_generators(
&mut blitzar_commitments,
&slice_ops::slice_cast(committable_columns),
&setup[offset..offset + max_column_len],
);
```

**최적화 특징:**

1. **배치 처리**: 여러 컬럼을 동시에 처리하여 GPU 활용률 극대화
2. **메모리 최적화**: 최대 길이 기준으로 생성원 할당하여 메모리 낭비 최소화
3. **타입 변환**: `slice_cast` 를 통한 제로카피 타입 변환
4. **GPU 가속**: CUDA 커널을 통한 대규모 병렬 연산

## 성능 벤치마크 결과

### GPU 초기화 성능

| 단계 | 일반적인 소요 시간 | 설명 |
|---|---|---|
| GPU 초기화 | ~520ms | 최초 1회만 필요, CUDA 커널 로딩 |
| 데이터 로딩 | ~3ms | 테이블 크기에 비례 |
| 쿼리 파싱 | ~2ms | 쿼리 복잡도에 비례 |
| 증명 생성 | 가변적 | 데이터 크기와 쿼리 복잡도에 따라 결정 |

### 하드웨어 요구사항

**권장 사양:**

- **GPU**: NVIDIA A100 (벤치마크 기준)
- **CUDA**: 11.0 이상
- **메모리**: 최소 8GB VRAM
- **드라이버**: 최신 NVIDIA 드라이버

**지원 환경:**

- **운영체제**: Linux x86_64 (주요 지원)
- **대안**: CPU 백엔드 ( `BLITZAR_BACKEND=cpu` )
- **개발 환경**: `--no-default-features --features="arrow cpu-perf"`

# CPU 백엔드 대안

## 환경 변수 설정

```
# CPU 백엔드 사용
export BLITZAR_BACKEND=cpu
cargo test --all-features --all-targets
```

```
# GPU 기능 완전 비활성화
cargo test --no-default-features --features="arrow cpu-perf"
```

## CPU 구현 특징

```
#[cfg(any(not(feature = "blitzar"), test))]
```

```rust
#[tracing::instrument(
name = "compute_commitment_generic_impl (cpu)",
level = "debug",
skip_all
)]
fn compute_commitment_generic_impl<T: Into<BNScalar> + Clone + Sync>(
setup: HyperKZGPublicSetup<'_>,
offset: usize,
scalars: &[T],
) -> HyperKZGCommitment {
assert!(offset + scalars.len() <= setup.len());
let product: G1Projective = if_rayon!(scalars.par_iter(), scalars.iter())
.zip(&setup[offset..offset + scalars.len()])
.map(|(t, s)| *s * Into::<BNScalar>::into(t).0)
.sum();
HyperKZGCommitment {
commitment: G1Projective::from(product),
}
}
```

**CPU 구현 특징:**

1. **Rayon 병렬화**: CPU 코어를 활용한 병렬 처리
2. **메모리 효율성**: GPU 메모리 할당 없이 시스템 RAM 사용

3. **호환성**: GPU가 없는 환경에서도 동작

4. **성능**: GPU 대비 느리지만 안정적인 성능 제공

# 오류 처리 및 디버깅

## 일반적인 GPU 오류

```
// GPU 초기화 실패 시 패닉

fn main() {

#[cfg(feature = "blitzar")]

{

let timer = start_timer("Warming up GPU");

proof_of_sql::base::commitment::init_backend(); // 여기서 패닉 가능

end_timer(timer);

}

}
```

**일반적인 오류 상황:**

- **CUDA 드라이버 미설치**: “CUDA driver not found” 오류
- **GPU 메모리 부족**: “out of memory” 오류
- **호환되지 않는 GPU**: “device not supported” 오류
- **권한 문제**: “permission denied” 오류

## 디버깅 도구

```
#[tracing::instrument(name = "compute_commitments (gpu)", level = "debug", skip_
```

**추적 기능:**

- **성능 측정**: 각 GPU 연산의 소요 시간 추적
- **메모리 사용량**: GPU 메모리 할당/해제 모니터링
- **오류 위치**: 정확한 오류 발생 지점 식별
- **배치 크기**: 최적 배치 크기 결정을 위한 데이터 수집

## 결론

Proof of SQL의 GPU 가속 시스템은 Blitzar 프레임워크를 통해 암호학적 연산의 성능을 대폭 향상시킵니다. 특히 commitment 계산에서 GPU의 병렬 처리 능력을 최대한 활용하여 실용적인 영지식 증명을 가능하게 합니다.

핵심 성과:

- **초기화 시간**: ~520ms (1회성)
- **병렬 처리**: 수천 개의 CUDA 코어 활용
- **메모리 최적화**: 효율적인 GPU 메모리 관리
- **호환성**: CPU 백엔드 대안 제공

이러한 GPU 가속 시스템은 Proof of SQL이 대규모 데이터셋에 대해 실시간 영지식 증명을 생성할 수 있게 하는 핵심 기술입니다.